LeetCode Solutions

LeetCode Assigned Problems

Name: Muhammad Farhan
Roll no: 25K-0538
Section: BCS-1K
Course: Programming Fundamentals Lab
Course Instructor: Sir Sheeraz Iqbal

QUESTION 1 (important)

Write a C program that creates a **2D dynamic array with 10 rows**.
The number of rows is fixed (10), but the **number of columns for each row must be decided by the user**, one row at a time.

Your program must:

1. Ask the user **10 times** to input an integer n.
    - n represents how many columns will be allocated for that row.
2. Dynamically allocate exactly n integers for that row.
3. Fill the entire row with the **row index**, where indices start from 1.
    - Row 1 should be filled with 1
    - Row 2 should be filled with 2
    - ...
    - Row 10 should be filled with 10
4. After all 10 inputs are processed, print the entire 2D array.
5. Make sure memory is properly freed.

Example Interaction

Input:
5
3
4
…
Output:
Row 1 → 1 1 1 1 1
Row 2 → 2 2 2
Row 3 → 3 3 3 3
…

```
#include <stdio.h>
#include <stdlib.h>

int main()
{

	int rows=10;
	int** array = (int**)malloc(rows * sizeof(int*));
	int* cols = (int*)malloc(rows * sizeof(int));

	for (int i = 0; i < rows; ++i)
	{
		printf("\nInput number of colums for row %d: ", i+1);
		scanf("%d", &cols[i]);
		array[i] = (int*)malloc(cols[i] * sizeof(int));
	}



	for (int i = 0; i < rows; ++i)
	{
		for (int j = 0; j < cols[i]; ++j)
		{
		array[i][j] = i+1;
		}
	}
	printf("\n\n");
```

```
    for (int i = 0; i < rows; ++i)
    {
        for (int j = 0; j < cols[i]; ++j)
        {
            printf("%d  ", array[i][j]);
        }
        printf("\n");
    }
    for (int i = 0; i < rows; ++i)
    {
        free(array[i]);
    }
    free(array);
    free(cols);
    array = NULL;
    cols = NULL;
    return 0;

}
```

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{

    int rows=10;
    int** array = (int**)malloc(rows * sizeof(int*));
    int* cols = (int*)malloc(rows * sizeof(int));

    for (int i = 0; i < rows; ++i)
    {
        printf("\nInput number of colums for row %d: ", i+1);
        scanf("%d", &cols[i]);
        array[i] = (int*)malloc(cols[i] * sizeof(int));
    }


    for (int i = 0; i < rows; ++i)
    {
        for (int j = 0; j < cols[i]; ++j)
        {
        array[i][j] = i+1;
        }
    }
    printf("\n\n");
    for (int i = 0; i < rows; ++i)
    {
        for (int j = 0; j < cols[i]; ++j)
        {
            printf("%d  ", array[i][j]);
        }
        printf("\n");
    }
    for (int i = 0; i < rows; ++i)
    {
        free(array[i]);
    }
    free(array);
    free(cols);
    array = NULL;
    cols = NULL;
    return 0;

}
```

```
Input number of colums for row 1: 4

Input number of colums for row 2: 3

Input number of colums for row 3: 5

Input number of colums for row 4: 7

Input number of colums for row 5: 2

Input number of colums for row 6: 3

Input number of colums for row 7: 4

Input number of colums for row 8: 5

Input number of colums for row 9: 6

Input number of colums for row 10: 2


1  1  1  1
2  2  2
3  3  3  3  3
4  4  4  4  4  4  4
5  5
6  6  6
7  7  7  7
8  8  8  8  8
9  9  9  9  9  9
10  10
[Finished in 9.51s]
```

QUESTION 2

Q3: Consider the code snippet below and answer the questions that follow accordingly.
[Marks 10, 2.5 each, 30 min]

```
int **create_matrix(int m, int n){
  int **p, i;
  p = (int**)malloc(sizeof(int*)*m);

for (int i = 0; i < m; ++i){
  p[i] = (int*)malloc(sizeof(int)*n);}

  return p;
}

void destroy_matrix(int **p){
  free(p);
}
```

(i) If m is 5 and n is 4 how many function calls to malloc will be made in the function above?

(ii) What and where if any error checking should have been performed in the create_matrix function?

(iii) The code compiles and runs, but gives memory leakage. What is the fault and how can it be corrected?

(iv) Write an updated version of create_matrix that creates matrixes for any values of m and n with just 2 calls to malloc function and despite that **p can be used as 2D array.

1. 6 function calls to Malloc will be made.
2. Adding the if (p == NULL) just below the p = (int**)malloc(sizeof(int*)*m); and (p[i] == NULL) inside the loop, below the p[i] = (int*)malloc(sizeof(int)*n), both conditions with warning, freeing all memory and then exit(1); should have been added in order to make sure none of the memory allocation had failed.
3. There is a memory leak because in the destroy_matrix function, we only freed the spine/array which holds each row together, we didn't free the rows. It can be corrected by first looping and freeing each and every row, before finally freeing the spine/frame.
4. The Code and screenshot is below:

```
#include <stdio.h>
#include <stdlib.h>

int** create_matrix(int m, int n);
void destroy_matrix(int** p);

int main()
{
	int rows, cols;
	int** array = NULL;
printf("\nEnter number of Rows for array: ");
scanf("%d", &rows);
printf("\nEnter number of Columns for array: ");
scanf("%d", &cols);
array = create_matrix(rows, cols);

destroy_matrix(array);
return 0;
}

int** create_matrix(int m, int n)
{
	int **p = (int**)malloc(m*sizeof(int*));
	if (p == NULL)
	{
		printf("\nMalloc Failed.");
```

```
		exit(1);
	}
	int* ptr = (int*)malloc(m*n*sizeof(int));
	if (ptr == NULL)
	{
		printf("\nMalloc Failed.");
		free(p);
		exit(1);
	}
	for (int i = 0; i < m; ++i)
	{
		p[i] = &ptr[i*n];
	}
	return p;
}

void destroy_matrix(int** p)
{
	free(p[0]);
	free(p);
}
```

```c
#include <stdio.h>
#include <stdlib.h>

int** create_matrix(int m, int n);
void destroy_matrix(int** p);

int main()
{
    int rows, cols;
    int** array = NULL;
printf("\nEnter number of Rows for array: ");
scanf("%d", &rows);
printf("\nEnter number of Columns for array: ");
scanf("%d", &cols);
array = create_matrix(rows, cols);

destroy_matrix(array);
return 0;
}

int** create_matrix(int m, int n)
{
    int **p = (int**)malloc(m*sizeof(int*));
    if (p == NULL)
    {
        printf("\nMalloc Failed.");
        exit(1);
    }
    int* ptr = (int*)malloc(m*n*sizeof(int));
    if (ptr == NULL)
    {
        printf("\nMalloc Failed.");
        free(p);
        exit(1);
    }
    for (int i = 0; i < m; ++i)
    {
        p[i] = &ptr[i*n];
    }
    return p;
}

void destroy_matrix(int** p)
{
    free(p[0]);
    free(p);
}
```

QUESTION 3

Implement a "**C program**" that dynamically allocates memory for strings and concatenates them. The program should perform the following steps:

1. **Input:**
   - Prompt the user to enter two strings of varying lengths.
   - Use dynamic memory allocation to create char arrays to store the input strings.
2. **Functionality:**
   - Create a function that takes the two input strings and dynamically allocates memory to concatenate them into a new string.
   - The concatenated string should have sufficient space for the combined strings and the null-terminator.
   - Repetitively take user inputs and concatenate until the users stops it with 'Q'.
   - All new inputs must be concatenated with the previous data. Do not over-write previous data in the variables. (Hint: Something related to ***re-allocation*** might help)
3. **Output:** Display the original input strings and the concatenated result.
4. **Error Handling:** Implement appropriate error handling. Check for memory allocation failures and inform the user if there's an issue.
5. **Testing:** Test your program with strings of different lengths to ensure correct memory allocation, concatenation, and freeing of memory.

```
#include <stdio.h>

#include <string.h>

#include <stdlib.h>


int main()

{

        int capacity = 2;

        int count = 0;

        char** storage = (char**)calloc(capacity, sizeof(char*));

        if (storage == NULL)

        {

                printf("\nStorage memory allocation failed.");

                exit(1);

        }
```

```
char* concat = NULL;
char* tempo = NULL;
char** tempoo = NULL;
char temp[20];
int size;
printf("\nEnter your first string: ");
scanf("%s", temp);
size = strlen(temp)+1;
concat = (char*)malloc(size*sizeof(char));
if (concat == NULL)
{
	printf("\nConcate memory allocation failed.");
	free(storage);
	exit(1);
}
tempo = (char*)malloc(size*sizeof(char));
if (tempo == NULL)
{
	printf("\nStorage First string Malloc failed.");
	exit(1);
}
storage[count] = tempo;
tempo = NULL;
strcpy(storage[count], temp);
strcpy(concat, temp);
count++;
```

```
while(1)
{
        printf("\nEnter new String or press Q to exit: ");
        scanf("%s", temp);

        if (strcmp(temp, "Q")== 0)
        {
                break;
        }

        size = strlen(temp);

        tempo = (char *)realloc(concat, strlen(concat)+size+1);
        if (tempo == NULL)
        {
                printf("\nConcate Realloc Failed");
                break;
        }
        concat = tempo;
        tempo = NULL;
        if (count >= capacity)
        {
                tempoo = (char**)realloc(storage, sizeof(char*)*(capacity+1));
                if (tempoo == NULL)
                {
                        printf("\nStorage Realloc Failed.");
```

```
                    break;
                }
                storage = tempoo;


                tempoo = NULL;
                capacity++;
            }
            tempo = (char*)malloc((size+1)*sizeof(char));
            if (tempo == NULL)
            {
                printf("\nNew Storage string Malloc failed.");
                break;
            }
            storage[count] = tempo;
            tempo = NULL;
            strcat(concat, temp);
            strcpy(storage[count], temp);
            count++;
        }
        printf("\n\nOriginal String inputs:\n");
        for (int i = 0; i < count; ++i)
        {
            printf("%s\n", storage[i]);
        }


        printf("\nFinal Concatenated String:\n");
```

```
	printf("\n%s", concat);


	for (int i = 0; i < count; ++i)
		{
			free(storage[i]);
		}
	free(storage);
	free(concat);
	return 0;
}
```

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int main()
{
    int capacity = 2;
    int count = 0;
    char** storage = (char**)calloc(capacity, sizeof(char*));
    if (storage == NULL)
    {
        printf("\nStorage memory allocation failed.");
        exit(1);
    }
    char* concat = NULL;
    char* tempo = NULL;
    char** tempoo = NULL;
    char temp[20];
    int size;
    printf("\nEnter your first string: ");
    scanf("%s", temp);
    size = strlen(temp)+1;
    concat = (char*)malloc(size*sizeof(char));
    if (concat == NULL)
    {
        printf("\nConcate memory allocation failed.");
        free(storage);
        exit(1);
    }
    tempo = (char*)malloc(size*sizeof(char));
    if (tempo == NULL)
    {
        printf("\nStorage First string Malloc failed.");
        exit(1);
    }
    storage[count] = tempo;
    tempo = NULL;
    strcpy(storage[count], temp);
    strcpy(concat, temp);
    count++;
    while(1)
    {
        printf("\nEnter new String or press Q to exit: ");
        scanf("%s", temp);

        if (strcmp(temp, "Q")== 0)
        {
            break;
        }

        size = strlen(temp);

        tempo = (char *)realloc(concat, strlen(concat)+size+1);
```

```c

        size = strlen(temp);

        tempo = (char *)realloc(concat, strlen(concat)+size+1);
        if (tempo == NULL)
        {
            printf("\nConcate Realloc Failed");
            break;
        }
        concat = tempo;
        tempo = NULL;
        if (count >= capacity)
        {
            tempoo = (char**)realloc(storage, sizeof(char*)*(capacity+1));
            if (tempoo == NULL)
            {
                printf("\nStorage Realloc Failed.");
                break;
            }
            storage = tempoo;

            tempoo = NULL;
            capacity++;
        }
        tempo = (char*)malloc((size+1)*sizeof(char));
        if (tempo == NULL)
        {
            printf("\nNew Storage string Malloc failed.");
            break;
        }
        storage[count] = tempo;
        tempo = NULL;
        strcat(concat, temp);
        strcpy(storage[count], temp);
        count++;
    }
    printf("\n\nOriginal String inputs:\n");
    for (int i = 0; i < count; ++i)
    {
        printf("%s\n", storage[i]);
    }

    printf("\nFinal Concatenated String:\n");
    printf("\n%s", concat);

    for (int i = 0; i < count; ++i)
        {
            free(storage[i]);
        }
    free(storage);
    free(concat);
    return 0;
}
```

```
Enter your first string: Hello

Enter new String or press Q to exit: World

Enter new String or press Q to exit: And

Enter new String or press Q to exit: Hello

Enter new String or press Q to exit: Sir

Enter new String or press Q to exit: Sheeraz

Enter new String or press Q to exit: I

Enter new String or press Q to exit: Hope

Enter new String or press Q to exit: You

Enter new String or press Q to exit: Are

Enter new String or press Q to exit: Having

Enter new String or press Q to exit: A

Enter new String or press Q to exit: Great

Enter new String or press Q to exit: Day

Enter new String or press Q to exit: !

Enter new String or press Q to exit: Q


Original String inputs:
Hello
World
And
Hello
Sir
Sheeraz
I
Hope
You
Are
Having
```

```
Original String inputs:
Hello
World
And
Hello
Sir
Sheeraz
I
Hope
You
Are
Having
A
Great
Day
!

Final Concatenated String:

HelloWorldAndHelloSirSheerazIHopeYouAreHavingAGreatDay!
[Finished in 335.31s]
```